

Introduction to Stata

Marco Rosso*

First version: September 2024

This version: September 28, 2026

[Click here for the latest version](#)

*University of Bologna; University of Cagliari; and CRENoS (email: marco.rosso4@unibo.it)

Preface	1
1 Introduction	2
1.1 What is Stata	2
1.2 The Reproducible Stata Workflow	3
2 Syntax, Operators and Functions	7
2.1 The Syntax of Stata Commands	7
2.2 Operators	8
2.3 Missing Values	9
2.4 Functions	10
2.5 Display	11
2.6 Repeating a Command by Groups: <code>by</code> and <code>bysort</code>	11
2.7 Quietly and Capture	12
Exercises	13
3 Loading, Inspecting and Cleaning Data	14
3.1 Loading and Saving Stata Datasets	14
3.2 Importing and Exporting Other Formats	15
3.3 Inspecting the Data	15
3.4 Checking the Data	16
3.5 Variables: Names, Types and Labels	17
3.6 Converting Between Strings and Numbers	18
3.7 Dates	18
3.8 Drop and Keep	19
3.9 Sort and Gsort	19
Exercises	20
4 Creating and Modifying Variables	21
4.1 Generate and Replace	21
4.2 Extended Generate: <code>egen</code>	22
4.3 Recode	22
4.4 Dummy Variables	23
4.5 Factor Variables	23
Exercises	25

5	Combining and Reshaping Data	26
5.1	Preserve, Restore and Temporary Files	26
5.2	Append	26
5.3	Merge	27
5.4	Reshape	28
5.5	Collapse	29
5.6	Contract	29
5.7	Expand	30
5.8	Fillin	30
	Exercises	31
6	Descriptive Statistics and Tests	32
6.1	Summarize	32
6.2	Frequency Tables: Tabulate	33
6.3	Tables of Statistics: Tabstat and Table	33
6.4	Means and Confidence Intervals	34
6.5	Percentiles and Cumulative Distribution	35
6.6	Correlation	35
6.7	Tests on Means and Proportions	35
	Exercises	37
7	Graphs	38
7.1	Histograms and Densities	38
7.2	Twoway Graphs: Relationships Between Variables	38
7.3	Bar Charts	39
7.4	Titles, Axes and Legends	39
7.5	Graphs of Estimation Results	40
7.6	Saving, Combining and Exporting Graphs	40
	Exercises	41
8	Linear Regression	42
8.1	OLS Regression	42
8.2	Reading the Output	42
8.3	Interpreting the Coefficients	43
8.4	Standard Errors	44
8.5	Predicted Values and Residuals	44
8.6	Hypothesis Testing After Estimation	45
8.7	Diagnostics	45
8.8	Stored Results	46
8.9	Publication-Ready Tables	46
	Exercises	48
9	Binary Outcome Models	49
9.1	Linear Probability Model	49
9.2	Logit and Probit	49
9.3	Marginal Effects: <code>margins</code>	50
9.4	Odds Ratios	51
9.5	Comparing the Three Models	51
	Exercises	52

10 Instrumental Variables	53
10.1 Two-Stage Least Squares: <code>ivregress</code>	53
10.2 What 2SLS Does	54
10.3 Post-Estimation Tests	54
Exercises	56
11 Panel Data	57
11.1 Declaring and Describing a Panel	57
11.2 Time-Series Operators	57
11.3 Fixed and Random Effects	58
Exercises	60
12 Macros, Loops and Stored Results	61
12.1 Macros	61
12.2 Loops	62
12.3 Stored Results	63
12.4 Conditional Execution: the <code>if</code> Command	63
12.5 Debugging	64
Exercises	65
A Common Errors	66
References	68

These notes, which are continuously updated¹, provide an introduction to the use of Stata for data analysis. They are designed for the Econometrics course in the Bachelor program, as Stata is the reference software for this course and is widely used in economics and in the other social sciences.

The course emphasizes the specification of an econometric model, the interpretation of model parameters, and the evaluation of model performance, rather than programming in Stata. However, code to obtain estimates and details on how to run the software are provided, enabling students to write their own code, under their own responsibility, for applications that may differ from those considered here.

How to use these notes

- Chapters 1–7 cover the tools needed to organize a project, load, clean and describe data, and produce graphs. Chapters 8–11 cover estimation (OLS, binary outcome models, instrumental variables, panel data). Chapter 12 introduces macros, loops and stored results, which make code shorter and less error-prone. Appendix A lists the most common error messages and how to fix them.
- Every example uses datasets that are shipped with Stata (**sysuse**) or downloadable from the Stata website (**webuse**), so that every line of code can be run as it is. Code boxes can be copied directly from the PDF into the do-file editor.
- Syntax diagrams are shown in *italics*. In syntax diagrams, elements between square brackets [] are optional, and | separates alternatives.
- The notes accompany two do-files: **handbook_examples.do**, which runs all the examples chapter by chapter, and **solutions.do**, which contains the solutions to the exercises at the end of each chapter.
- The notes are written for Stata 18 or later. Most commands also work in earlier versions; the few that do not are flagged in the text.

¹Always check that you are using the latest version.

1.1 What is Stata

Stata is a statistical software package for data management, statistical analysis and graphics. It can be used both through menus (point-and-click) and through commands. Commands are what make an analysis **reproducible**: a sequence of commands saved in a file can be run again, by you or by someone else, and it will produce exactly the same results. For this reason, in these notes we always work with commands.

Stata handles datasets organized as matrices, with variables represented in columns and observations in rows. Variables can be numeric or strings (text). Stata offers a complete set of tools for descriptive statistics, statistical tests, linear and nonlinear regression, instrumental variables, panel data, time series, maximum likelihood, simulation and bootstrap, and many user-written extensions are freely available.

Stata is available in different editions, which differ in the size of the datasets they can handle and in speed, not in the commands they offer:

- Stata/BE (Basic Edition): suitable for moderate-sized datasets¹;
- Stata/SE: designed for larger datasets²;
- Stata/MP: the fastest edition, optimized for multicore computers.

Stata runs on Windows, macOS and Linux. The commands are the same on all platforms.

Upon starting a new Stata session, four default windows are typically opened:

- **command window**: commands can be directly typed into this window for execution;
- **results window**: this window displays the output generated by executed commands;
- **history (review) window**: this window keeps a list of all executed commands;
- **variables window**: this window lists the names and labels of all variables currently loaded in memory. The **properties window** below it shows the details of the selected variable.

Two additional windows are used very frequently:

¹Called Stata/IC up to version 16.

²The University of Bologna provides Stata to its students free of charge. Check the university software portal for the current edition and the installation instructions.

- **do-file editor**: this window is used to create and edit do-files, which contain sequences of Stata commands that can be executed as a batch (command: `doedit`);
- **viewer**: this window displays help files and log files; the **data editor/browser** (commands: `edit`, `browse`) shows the data as a spreadsheet.

note: use `browse` to look at the data, never `edit` to change them by hand: changes made by hand are not recorded anywhere and cannot be reproduced.

1.2 The Reproducible Stata Workflow

To ensure all analysis is reproducible, Stata users must adopt a workflow centered on two key files: the **do-file** and the **log-file**.

The Working Directory

Stata reads and writes files in a folder called the **working directory**. If no path is specified, data and files are read from and written to this folder. The working directory can be checked and changed using the commands:

- `pwd`: displays the current working directory (print working directory);
- `cd "path_to_my_project"`: changes the current working directory.

When Stata is opened from the menu or the dock, the working directory is usually your Documents folder. When Stata is opened by double-clicking a do-file, the working directory is usually the folder that contains that do-file. Always check it with `pwd`.

tips: always use forward slashes `/` in paths, also on Windows (e.g., `"C:/Users/name/project"`). Stata accepts them on every operating system, while a backslash `\` placed before a macro (see Chapter 12) prevents the macro from being expanded.

The Do-File: Your Command Script

The **do-file** (`.do` extension) is a text file that contains all the commands necessary to reproduce your analysis, from loading the data to producing the final tables and graphs.

- The do-file editor is opened from the menu or with the command `doedit`. Commands are executed by pressing the “Do” button (the whole file) or by highlighting some lines and pressing “Do” (only the selected lines). The keyboard shortcut is `Ctrl+D` on Windows and `Cmd+Shift+D` on macOS.
- A do-file can also be executed from another do-file with `do "filename.do"`.

There are several ways to write comments in a do-file:

- a line starting with `*` is a comment;
- everything after `//` is a comment, so `//` can also be used after a command on the same line;
- everything between `/*` and `*/` is a comment, even over several lines;

- `///` at the end of a line tells Stata that the command continues on the next line. It is used to split long commands.

```
* This is a comment
summarize price mpg      // this is a comment after a command
/* This comment
   spans several lines */
regress price mpg weight ///
    foreign, vce(robust) // one command written on two lines
```

note: `//` and `///` are not interchangeable. `//` turns the rest of the line into a comment, so a command split with `//` stops where `//` is. Moreover, `//`, `///` and `/* */` only work in do-files, not in the command window.

The Log-File: Capturing Output

The **log-file** captures all the output displayed in the results window, including tables and estimation results. It is the record of what your code produced.

- `log using name.log, text replace`: starts recording the output to a plain-text file, replacing it if it already exists (without `text`, Stata uses its own `.smcl` format, which can only be read in Stata's viewer);
- `log using name.log, text append`: continues writing to an existing log-file;
- `log close`: stops recording the output.

You can also keep several log-files open at the same time, provided that each of them has a name:

- `log using name_1.log, text replace name(log_1)`: starts recording the output in the first file;
- `log using name_2.log, text replace name(log_2)`: starts recording the output in a second file, while the first one stays open;
- `log close log_1`: stops recording the output in the named log-file;
- `log close _all`: stops recording in all log-files.

Organizing Your Project

It is also important to organize the project folders well. A recommended practice is to create a main project folder, with sub-folders for specific file types:

- **Dataset**: the data used in the project. Original (raw) data should never be overwritten: save modified data under a different name;
- **Output**: tables and other output files;
- **Graphs**: graphs and figures;
- **Logs**: log-files;

- **Do-files** (optional): the do-files can also be stored in the main folder.

The only information that depends on the computer is the location of the main folder (the **root**). If this information is stored in a single place, and all other paths are written relative to it, the code runs on any computer. We store the root in a **global macro** (see Chapter 12), a name that Stata replaces with its content every time it finds **\$name** in the code.

The template below goes one step further: by default the root is the current working directory, so that the code runs without modifications if the do-file is launched from the main folder, and the sub-folders are created automatically if they do not exist.

```

=====
* Example of how to organize a new do-file
=====
version 18
clear all
capture log close _all
set more off                // default since Stata 16

*-----
* 1. Root directory
*-----
* By default, the root is the folder where Stata is working
* (the folder of this do-file, if you opened it by double-clicking)
global root "`c(pwd)'"
* To fix the path by hand, remove the * in the next line:
* global root "C:/Users/name/Documents/MyProject"

*-----
* 2. Sub-folders (created only if they do not exist)
*-----
foreach folder in Dataset Output Graphs Logs {
    capture mkdir "$root/`folder'"
}
global Dataset "$root/Dataset"
global Output  "$root/Output"
global Graphs  "$root/Graphs"
global Logs    "$root/Logs"

cd "$root"
display "Project folder: $root"

*-----
* 3. Log-file and data
*-----
log using "$Logs/log_name.log", text replace name(main)

use "$Dataset/dataset.dta", clear

* ... analysis ...

log close main

```

where:

- **version 18** tells Stata to interpret the code as Stata 18 would, so that the do-file gives the same results in future versions;
- **clear all** removes data and stored results from memory and **capture log close _all** closes any log-file left open by a previous run (**capture** avoids an error if no log is open);

- `c(pwd)` contains the current working directory, so the first `global root` line makes the code independent of the computer; the commented line is the alternative when you prefer to write the path explicitly;
- `capture mkdir` creates each sub-folder, and `capture` ignores the error returned when the folder already exists;
- the log-file is opened *before* loading the data, so that everything the code does is recorded.

note: with the default root, the code runs correctly only if Stata is working in the project folder. Either open Stata by double-clicking the do-file, or type `cd "path_of_the_project"` once in the command window before running it. The line `display "Project folder: $root"` lets you check where the files are being written.

Stata's Help System and Documentation

Stata's documentation is excellent and is your most reliable resource.

- `help command_name`: displays the syntax, options and examples of a command. At the top of each help file there is a link to the full PDF manual entry, which contains the methods and formulas;
- `search keyword`: searches the official documentation and online resources for a keyword;
- `ssc install package`: installs a user-written package from the SSC archive (see Chapter 8).

tips: most commands can be abbreviated to their first letters (e.g., `sum` for `summarize`, `tab` for `tabulate`, `reg` for `regress`). The minimum abbreviation is underlined in the syntax diagram of the help file.

Syntax, Operators and Functions

2.1 The Syntax of Stata Commands

Most Stata commands follow the same syntax:

```
[by varlist:] command [varlist] [=exp] [if exp] [in range] [weight] [, options]
```

where:

- **by varlist:** repeats the command for each group of observations defined by the values of the variables in **varlist** (see Section 2.6);
- **command** is the operation that Stata is instructed to perform (e.g., **summarize**, **list**, **regress**);
- **varlist** is the list of variables on which the command is executed. It is optional for many commands; when omitted, the command usually applies to all variables;
- **=exp** is an expression, used by commands that create or modify variables (e.g., **generate**);
- **if exp** restricts the command to the observations for which the expression is true;
- **in range** restricts the command to a range of observations (e.g., **in 1/10** for the first ten, **in -5/1** for the last five);
- **weight** specifies weights (see below);
- **, options** modify the behaviour of the command. Options always come after a single comma.

Variable lists can be written in a compact way:

- **price-weight:** all variables from **price** to **weight**, in the order in which they appear in the dataset;
- **inc*:** all variables whose name starts with **inc**;
- **_all:** all variables.

Weights

Many commands accept weights, written in square brackets after the variable list, e.g., `summarize wage [aweight = hours]`. There are four types:

- **fweight** (frequency weights): the number of identical observations that each row represents. They must be integers;
- **pweight** (sampling weights): the inverse of the probability that the observation was sampled. They are the weights used with survey data and imply robust standard errors;
- **aweight** (analytic weights): inversely proportional to the variance of the observation, typically used when each observation is an average over a group of different size;
- **iweight** (importance weights): used mainly by programmers.

Not all commands accept all types of weights: `help weight` and the help file of each command report which types are allowed.

2.2 Operators

Arithmetic Operators

- `+` addition, `-` subtraction, `*` multiplication, `/` division;
- `^` power (e.g., `age^2`).

String Operators

- `+` combines two strings into one, e.g., `"variance-" + "covariance matrix"` results in `"variance-covariance matrix"`;
- `*` repeats a string, e.g., `"ab"*3` results in `"ababab"`.

Relational Operators

These operators compare two values and return 1 if the comparison is true and 0 if it is false.

- `<` less than; `>` greater than;
- `<=` less than or equal to; `>=` greater than or equal to;
- `==` equal to;
- `!=` or `~=` not equal to.

note: `=` is used to assign a value (`generate x = 1`), `==` is used to compare two values (`keep if foreign == 1`). Writing `if foreign = 1` returns an error.

Logical Operators

- `&` (and) is true if both expressions are true;
- `|` (or) is true if at least one of the expressions is true;
- `!` or `~` (not) is true if the expression is false.

When operators are combined, Stata evaluates them in the following order: `!`, then `^`, then `*` and `/`, then `+` and `-`, then relational operators, then `&`, and finally `|`. When in doubt, use parentheses:

```
sysuse auto, clear
generate cheap_efficient = (price < 5000) & (mpg > 25)
count if (rep78 == 4 | rep78 == 5) & foreign == 1
```

The first line creates a variable that is 1 for cars that cost less than 5,000 dollars and do more than 25 miles per gallon, and 0 otherwise. The second counts the foreign cars with a repair record of 4 or 5.

2.3 Missing Values

A missing numeric value is displayed as a dot (`.`). Stata also has 26 *extended* missing values, `.a`, `.b`, ..., `.z`, which can be used to distinguish the reasons why a value is missing (e.g., “don’t know” versus “refused to answer” in a survey). A missing string value is an empty string (`""`).

Three rules are essential:

1. **In comparisons, a missing value is larger than any number:** `.` `>` any number, and `.a < .b < ... < .z`. Therefore if `income > 50000` is **also true for observations with missing income**.
2. Any arithmetic operation involving a missing value, or an impossible operation (e.g., division by zero, the logarithm of a negative number), returns a missing value.
3. Estimation commands and most statistics commands automatically exclude observations with missing values in any of the variables used (*listwise deletion*).

The first rule is the source of many silent errors. Compare:

```
sysuse auto, clear
count if rep78 > 3 // includes the 5 cars with missing rep78
count if rep78 > 3 & !missing(rep78) // correct
count if rep78 > 3 & rep78 < . // equivalent
generate good_record = (rep78 > 3) // missing rep78 coded as 1: WRONG
generate good_record2 = (rep78 > 3) if !missing(rep78) // correct
```

The function `missing(x)` returns 1 if `x` is missing (any of `.`, `.a`, ..., `.z`, or `""` for strings), and 0 otherwise; `!missing(x)` is its negation. To see how many values are missing for each variable, use:

```
misstable summarize
```

note: every time you write `>`, `>=` or `!=` in an `if` condition or in a `generate` command, ask yourself what happens to observations with missing values.

2.4 Functions

Functions are used within expressions and can take other functions as arguments. The complete list is available with `help functions`.

Mathematical Functions

- `exp(x)`: exponential of `x`;
- `ln(x)` or `log(x)`: **natural** logarithm of `x` (the two are synonyms);
- `log10(x)`: logarithm base 10 of `x`;
- `sqrt(x)`: square root; `abs(x)`: absolute value;
- `round(x)`: rounds to the nearest integer; `round(x, 0.1)`: rounds to the first decimal;
- `floor(x)` and `ceil(x)`: largest integer $\leq x$ and smallest integer $\geq x$;
- `int(x)`: integer part, obtained by truncating towards zero (`int(-2.7)` is `-2`);
- `max(x1, x2, ..., xn)` and `min(x1, x2, ..., xn)`: maximum and minimum among the arguments, ignoring missing values;
- `sum(x)`: **running** (cumulative) sum of `x` over the observations, treating missing values as zeros. It is not the total: for totals see `egen` in Chapter 4.

Statistical Functions

- `normal(z)`: cumulative distribution function of the standard normal, $\Phi(z)$;
- `normalden(z)`: density of the standard normal, $\phi(z)$;
- `invnormal(p)`: quantile of the standard normal (e.g., `invnormal(0.975) = 1.96`);
- `ttail(df, t)`: upper-tail probability of the Student's t with `df` degrees of freedom, $P(T > t)$. The two-sided p-value is `2*ttail(df, abs(t))`;
- `invttail(df, p)`: the value t such that $P(T > t) = p$ (e.g., the critical value at 5% two-sided is `invttail(df, 0.025)`);
- `chi2tail(df, x)` and `Ftail(df1, df2, f)`: upper-tail probabilities of the χ^2 and F distributions.

Random Numbers

- `runiform()`: pseudo-random numbers uniformly distributed on $[0, 1]$;
- `rnormal()` and `rnormal(m, s)`: pseudo-random numbers from a standard normal and from a normal with mean `m` and standard deviation `s`.

note: random numbers change every time the code is run. To make results reproducible, always set the seed before generating them: `set seed 12345`.

String Functions

- `strlen(s)`: number of characters; `substr(s, n1, n2)`: substring of `s` starting at position `n1` and of length `n2`;
- `strupper(s)`, `strlower(s)`: upper and lower case; `strtrim(s)`: removes leading and trailing blanks;
- `strpos(s1, s2)`: position of `s2` in `s1` (0 if not found);
- `subinstr(s1, s2, s3, .)`: replaces all occurrences of `s2` in `s1` with `s3`;
- `word(s, n)`: the `n`-th word of `s`;
- `real(s)`: converts a string to a number; `string(n)`: converts a number to a string.

Let's see some examples:

```
sysuse auto, clear
generate ln_price = ln(price)
generate price_k  = round(price/1000, 0.1)    // price in thousands, 1 decimal
generate brand    = word(make, 1)             // first word of the car name
set seed 12345
generate u = runiform()
generate z = rnormal()
display 2*ttail(72, abs(2.1))                  // two-sided p-value of t = 2.1
```

2.5 Display

The `display` command shows strings and the values of expressions, and can be used as a calculator:

```
display ["string"] [exp] [...]
```

For example:

- `display "this is a sentence"` displays the string in the results window;
- `display 5 + ln(70)` computes and displays `5 + ln(70)`;
- `display invnormal(0.975)` displays the 97.5% quantile of the standard normal;
- `display "The mean price is " r(mean)` displays a string followed by a stored result (see Chapter 12).

2.6 Repeating a Command by Groups: `by` and `bysort`

The `by` prefix repeats a command for each group of observations defined by one or more variables. The data must be sorted by those variables; `bysort` sorts and executes in one step.

```
by varlist: command
bysort varlist1 [(varlist2)]: command
```

where `varlist1` defines the groups and the optional `varlist2`, in parentheses, defines the order of the observations within each group, without defining new groups.

```
sysuse auto, clear
bysort foreign: summarize price mpg      // statistics separately by origin
```

Inside `by`, two built-in variables refer to the group rather than to the whole dataset:

- `_n`: the position of the observation within the group (1, 2, 3, ...);
- `_N`: the number of observations in the group.

This makes `by` essential whenever the result depends on the other observations of the group. For example:

```
bysort foreign: generate n_cars = _N          // number of cars by origin
bysort foreign (price): generate cheapest = (_n == 1) // cheapest car by origin
list make foreign price if cheapest == 1
```

The subscript `x[_n-1]` refers to the value of `x` in the previous observation. With panel data, sorting by unit and time and using `by` gives the lagged value within each unit:

```
webuse nlswork, clear
bysort idcode (year): generate lag_wage = ln_wage[_n-1]
```

For the first year of each individual, `lag_wage` is missing, as it should be. Without `by`, the first observation of each individual would take the value of the last observation of the previous individual. Chapter 11 shows a safer way to create lags, which also takes gaps in the years into account.

note: `by` is only useful when the command depends on the group. `bysort foreign: drop if price > 10000` gives exactly the same result as `drop if price > 10000`.

2.7 Quietly and Capture

- `quietly` command executes the command without displaying its output. The output is also **not** written in the log-file. It is useful when only the stored results are needed:

```
sysuse auto, clear
quietly summarize price
display "Mean price: " r(mean)
```

- `capture` command executes the command and, if it returns an error, continues the do-file instead of stopping. The error code is stored in `_rc` (0 if there was no error). Use it only when an error is expected and harmless (e.g., `capture log close`, `capture mkdir`), otherwise real errors go unnoticed.

Exercises

1. Load `auto.dta`. Use `display` to compute $\ln(100)$, $\log_{10}(100)$ and the 95% two-sided critical value of a t distribution with 70 degrees of freedom.
2. Count the cars with `rep78` greater than or equal to 4. How many would you count if you forgot about missing values?
3. Create a variable `heavy` equal to 1 if `weight` is above 3,000 pounds and 0 otherwise, and a variable `heavy_good` equal to 1 for heavy cars with a repair record of 4 or 5, 0 for the other cars, and missing when `rep78` is missing.
4. Create a variable containing the first word of `make` in upper case.
5. Using `bysort`, create a variable equal to the most expensive price within each value of `rep78`, and list the most expensive car for each value.
6. Set the seed to 2026 and generate a variable drawn from a normal distribution with mean 10 and standard deviation 2. Check its mean and standard deviation with `summarize`.

Loading, Inspecting and Cleaning Data

3.1 Loading and Saving Stata Datasets

Data files in Stata are saved with the extension `.dta`. To load a dataset:

```
use "path/filename.dta" [, clear]
```

The `clear` option removes the dataset currently in memory, if any. Without it, Stata refuses to load a new dataset when the data in memory have changed since they were last saved. The extension `.dta` can be omitted.

Stata also includes several example datasets, which are useful for learning and experimenting with commands. The datasets installed with Stata are loaded with `sysuse`, while those used in the manuals are downloaded from the Stata website with `webuse` (an internet connection is required):

```
sysuse auto, clear           // 1978 car models
sysuse nlsw88, clear         // 1988 wages of US women
webuse nlswork, clear        // panel of US women, 1968-1988
sysuse dir                   // list of the datasets installed with Stata
```

To save the data in memory:

```
save "path/filename.dta" [, replace]
```

The `replace` option is needed to overwrite an existing file. Without it, Stata returns an error if the file already exists, which protects you from overwriting a file by mistake.

note: never overwrite the original data. Load the raw data, clean them with a do-file, and save the result under a new name (e.g., `survey_clean.dta`). If something goes wrong, you can always start again from the raw data.

3.2 Importing and Exporting Other Formats

Delimited Text Files (CSV)

The `import delimited` command imports text files in which the columns are separated by a delimiter (comma, tab, semicolon, ...). By default, Stata detects whether the delimiter is a comma or a tab, and uses the first row as variable names if it looks like a row of names.

```
import delimited "$Dataset/external.csv", clear
import delimited "$Dataset/external_it.csv", delimiters(";") clear
import delimited "$Dataset/data.txt", delimiters(tab) varnames(1) clear
```

note: files produced by Excel with Italian settings use the semicolon as delimiter and the comma as decimal separator. Use `delimiters(";")` and `decimalseparator(,)`.

Excel Files

The `import excel` command imports spreadsheets and allows you to choose the sheet and the range of cells:

```
import excel "$Dataset/survey.xlsx", sheet("Sheet1") firstrow clear
import excel "$Dataset/survey.xlsx", sheet("Sheet1") cellrange(A3:F200) firstrow clear
```

The `firstrow` option tells Stata to treat the first row of the sheet (or of the range) as variable names rather than data. Without it, the variables are named after the Excel columns (A, B, C, ...).

Exporting

```
export delimited using "$Output/data.csv", replace
export excel using "$Output/data.xlsx", firstrow(variables) replace
```

3.3 Inspecting the Data

Describe

The `describe` command summarizes the dataset in memory, or a dataset stored on disk:

```
describe [varlist] [, short]
describe [varlist] using filename [, short]
```

It reports the number of observations and variables, and, for each variable, its storage type, display format, value label and variable label. The `short` option only reports the information on the dataset (number of observations and variables, size, date), without the list of variables.

Codebook

The `codebook` command gives a more detailed description of each variable: type, range, number of unique values, number of missing values, mean and percentiles for numeric variables,

frequencies for variables with few values.

```
sysuse auto, clear
describe
codebook rep78 foreign
```

List and Browse

The `list` command displays the values of the variables in the results window:

```
[by varlist:] list [varlist] [if exp] [in range] [, nolabel noobs]
```

where `nolabel` displays the numeric codes instead of the value labels, and `noobs` suppresses the observation numbers. For example:

- `list in 1/5` displays all variables for the first five observations;
- `list make price mpg` displays three variables for all observations;
- `list make price if foreign == 1 & price > 10000` displays the expensive foreign cars.

To look at the data as a spreadsheet, use `browse [varlist] [if exp]`.

Count

The `count` command counts the observations, optionally those that satisfy a condition:

```
[by varlist:] count [if exp] [in range]
```

```
count                                // number of observations
count if price > 10000
bysort foreign: count if mpg > 25    // separately for domestic and foreign cars
```

3.4 Checking the Data

Before any analysis, it is good practice to check that the data are what you think they are. Three commands are especially useful.

- `isid varlist` checks that the variables in `varlist` uniquely identify the observations. It returns an error if they do not. It should be run before every `merge` and `reshape` (Chapter 5).
- `duplicates report [varlist]` reports how many observations are repeated (identical in all variables, or in the variables of `varlist`); `duplicates list` lists them and `duplicates drop` drops them.
- `assert exp` checks that the expression is true for every observation and stops the do-file with an error if it is not. It turns your assumptions about the data into checks.

```
sysuse auto, clear
isid make // each car appears once
duplicates report make
assert price > 0 & !missing(price)
assert inlist(foreign, 0, 1)
```

3.5 Variables: Names, Types and Labels

In Stata, variables correspond to columns and observations to rows. Variable names can contain letters, digits and underscores, must start with a letter or an underscore, cannot contain spaces, and are case-sensitive (**Price** and **price** are different variables).

Variables are either **numeric** or **strings**. Numeric variables are stored as **byte**, **int**, **long** (integers) or **float**, **double** (decimals); **float** is the default. Identifiers with many digits (e.g., tax codes, long ID numbers) must be stored as **long**, **double** or strings, because a **float** has only about 7 significant digits. The command **compress** changes each variable to the smallest type that can store it without loss of information, reducing the size of the dataset.

Rename and Order

```
rename price price_usd // rename one variable
rename (mpg weight) (miles_gallon weight_lbs) // several variables at once
order make foreign // move variables to the beginning of the dataset
```

Variable Labels and Value Labels

A **variable label** describes the variable itself (e.g., “Price in US dollars”), while **value labels** map numeric codes to meaningful text (e.g., 0 to “Domestic”, 1 to “Foreign”). Labels make descriptive statistics, regression output and graphs readable.

```
sysuse auto, clear
label variable price "Price (USD)"
```

Value labels are assigned in two steps: first the map (the label set) is defined with **label define**, then it is attached to one or more variables with **label values**:

```
sysuse nlsw88, clear
generate area = 1 if south == 0
replace area = 2 if south == 1
label define area_lbl 1 "Non-South" 2 "South"
label values area area_lbl
label variable area "Geographic area"
tabulate area
label list area_lbl // shows the codes and their labels
```

Display Formats

The **format** command controls how a variable is displayed, without changing the values stored:

```
format varlist %fmt
```

For example:

- `format price %9.2f`: fixed format, 9 characters wide with 2 decimals;
- `format price %9.0fc`: no decimals, with commas as thousands separators;
- `format make %-20s`: string left-aligned in 20 characters;
- `format date %td`: displays a daily date (see Section 3.7).

3.6 Converting Between Strings and Numbers

Data imported from other formats often contain numbers stored as strings (displayed in red in the data browser). Statistical commands cannot use them.

- `destring varlist, replace` converts strings that contain numbers into numeric variables. If some values contain non-numeric characters (e.g., "1,000" or "n.a."), Stata refuses to convert; the option `ignore(",")` removes the listed characters first, and `force` turns non-numeric values into missing values (check what you are losing before using it);
- `tostring varlist, replace` converts numeric variables into strings;
- `encode strvar, generate(newvar)` converts a string *categorical* variable (e.g., region names) into a numeric variable with value labels. The codes 1, 2, 3, ... follow the alphabetical order of the strings;
- `decode numvar, generate(newvar)` does the opposite, creating a string from the value labels.

note: `destring` is for strings that *are* numbers ("1250"); `encode` is for strings that are *categories* ("Emilia-Romagna"). Using `encode` on numbers stored as strings produces codes that have nothing to do with the original values.

3.7 Dates

Stata stores daily dates as the number of days since 1 January 1960 and displays them with the format `%td`. Dates stored as strings must be converted:

```
clear
input str10 date_str
"25/12/2024"
"01/03/2025"
end
generate date = date(date_str, "DMY")    // day-month-year order
format date %td
generate year = year(date)
generate month = month(date)
list
```

The second argument of `date()` gives the order of day, month and year in the string ("DMY", "MDY", "YMD"). The functions `year()`, `month()`, `day()` and `dow()` (day of the week) extract the components of a date, and `mdy(m, d, y)` builds a date from its components.

3.8 Drop and Keep

The `drop` and `keep` commands remove variables or observations from the data in memory. `drop` specifies what to remove, `keep` what to retain.

```
drop varlist
drop if exp
drop in range
keep varlist
keep if exp
keep in range
```

For example:

- `drop rep78` drops the variable `rep78`;
- `keep make price mpg` keeps only these three variables;
- `drop if price > 10000` drops the observations with price above 10,000 (and those with missing price!);
- `keep if foreign == 1` keeps only the foreign cars;
- `drop in 1/5` drops the first five observations.

note: `drop` and `keep` cannot be undone, except by reloading the data. Before dropping observations, run `count` with the same condition to check how many you are going to lose.

3.9 Sort and Gsort

The `sort` command arranges the observations in ascending order of the variables in `varlist`; missing values are placed at the end.

```
sort varlist [, stable]
```

When several observations have the same values of `varlist` (ties), their order after `sort` is random and can change from one run to the next. The `stable` option keeps tied observations in their original order. This matters every time the result depends on the order, e.g., when using `_n`.

The `gsort` command also allows descending order:

```
gsort [+/-] varname [[+/-] varname ...] [, generate(newvar) mfirst]
```

where `-` before a variable sorts it in descending order, `generate(newvar)` creates a variable numbering the groups of identical values in the new order, and `mfirst` places missing values first when sorting in descending order. For example:

- `sort price` sorts by price in ascending order;
- `gsort -price` sorts by price in descending order;
- `gsort foreign -price` sorts by origin (ascending) and, within origin, by price (descending).

Exercises

1. Load `nls88.dta`. Use `describe` and `codebook` to find how many observations and variables the dataset has, and which variables have missing values.
2. List the first 10 observations for the variables `idcode`, `age`, `wage` and `industry`, first with the value labels and then with the numeric codes.
3. Check that `idcode` uniquely identifies the observations, and use `assert` to check that all wages are positive.
4. Create a variable `married_str` equal to the value label of `married` (`decode`), and then convert it back to a numeric variable with `encode`. Are the codes the same as in `married`? Why?
5. Keep only women aged 35 to 40 who are not missing `union`, and sort the data by `wage` in descending order. Who earns the most?
6. Save the resulting dataset as `nls88_sub.dta` in the `Dataset` folder of your project, and export it to a CSV file in the `Output` folder.

Creating and Modifying Variables

4.1 Generate and Replace

New variables are created with `generate` (abbreviated `gen`) and existing variables are modified with `replace`:

```
generate [type] newvar = exp [if exp] [in range]
replace oldvar = exp [if exp] [in range]
```

For example:

```
sysuse auto, clear
generate one = 1 // a column of 1s
generate price_k = price/1000 // price in thousands of dollars
generate ln_price = ln(price)
generate mpg2 = mpg^2
generate double id = _n // observation number, stored as double
generate make_upper = strupper(make) // string variables work the same way
```

When `generate` is used with `if`, the observations that do not satisfy the condition get a missing value. `replace` is then used to fill in the other observations:

```
generate size = 1 if length < 170
replace size = 2 if length >= 170 & length < 200
replace size = 3 if length >= 200 & !missing(length)
label define size_lbl 1 "Small" 2 "Medium" 3 "Large"
label values size size_lbl
tabulate size, missing
```

tips: after creating a variable from several conditions, always check the result with `tabulate newvar, missing` or with a cross-tabulation of the old and the new variable. `generate` and `replace` report how many missing values were generated and how many values were changed: read these messages.

4.2 Extended Generate: egen

The `egen` command (extended generate) provides functions that `generate` does not have, in particular functions that compute a statistic over several observations or over several variables:

```
[bysort varlist:] egen [type] newvar = function(arguments) [if exp] [in range] [, options]
```

Statistics over observations

Functions such as `mean()`, `median()`, `sd()`, `min()`, `max()`, `total()`, `count()` and `rank()` compute a statistic over the observations and assign it to **every** observation. Combined with `bysort`, they compute the statistic within each group:

```
sysuse auto, clear
egen mean_price = mean(price)           // overall mean, in every row
bysort foreign: egen mean_price_g = mean(price) // mean by origin
bysort foreign: egen n_price = count(price) // non-missing prices by origin
egen rank_mpg = rank(mpg)               // rank of each car's mileage
generate dev_price = price - mean_price_g // deviation from the group mean
```

Statistics over variables (row functions)

Functions such as `rowtotal()`, `rowmean()`, `rowmax()`, `rowmin()` and `rowmiss()` compute a statistic across variables, for each observation:

```
egen size_index = rowmean(length weight displacement)
egen n_missing = rowmiss(price mpg rep78)
```

note: `generate total = x1 + x2 + x3` returns a missing value if any of the three variables is missing, while `egen total = rowtotal(x1 x2 x3)` treats missing values as zeros. Neither is right or wrong in general: choose deliberately.

Group identifiers

`egen newvar = group(varlist)` creates a variable numbering the combinations of values of `varlist` (1, 2, 3, ...). It is useful to create a single identifier from several variables (e.g., region and year) or a numeric identifier from a string:

```
sysuse nlsw88, clear
egen group_id = group(race south)
tabulate group_id
```

4.3 Recode

The `recode` command changes the values of numeric variables according to rules, and is convenient to group the values of a variable into categories:

```
recode varlist (rule) [(rule) ...] [, generate(newvar)]
```

```
sysuse nlsw88, clear
recode grade (0/11 = 1 "Less than HS") (12 = 2 "High school") ///
           (13/15 = 3 "Some college") (16/18 = 4 "College +"), ///
           generate(educ_cat)
tabulate grade educ_cat
```

The `generate()` option creates a new variable and leaves the original unchanged, which is always preferable. Values not covered by any rule, including missing values, are left unchanged. To split a continuous variable into groups of equal size (quantiles), use `xtile`:

```
xtile wage_q = wage, nquantiles(4)      // quartiles of the wage distribution
tabulate wage_q
```

4.4 Dummy Variables

A dummy (indicator) variable takes value 1 if a condition is true and 0 otherwise.

Dummy for a specific condition

A relational expression returns 1 when true and 0 when false, so a dummy can be created in one line. Remember to handle missing values (Section 2.3):

```
sysuse nlsw88, clear
generate college = (grade >= 16) if !missing(grade)
generate union_south = (union == 1 & south == 1) if !missing(union, south)
```

Without `if !missing(grade)`, women with missing education would be coded as 1 in `college`, since a missing value is larger than 16. In the second line, `missing(union, south)` is 1 if any of the two variables is missing.

Dummies for all categories

The `generate()` option of `tabulate` creates one dummy for each category of a variable:

```
tabulate race, generate(race_d)      // creates race_d1, race_d2, race_d3
```

The dummies are missing when `race` is missing.

4.5 Factor Variables

In estimation commands, it is rarely necessary to create dummies by hand. **Factor-variable notation** tells Stata how to treat each variable directly in the command:

- `i.varname`: `varname` is categorical; Stata includes one dummy for each category except the base (reference) category, which by default is the lowest value;
- `ib#.varname`: uses the value `#` as base category (e.g., `ib2.race`);
- `ib(last).varname`, `ib(freq).varname`: use the largest or the most frequent value as base category;

- `c.varname`: `varname` is continuous (the default outside interactions);
- `a#b`: interaction between `a` and `b` only;
- `a##b`: main effects of `a` and `b` and their interaction (`a##b = a b a#b`).

```
sysuse nlsw88, clear
regress wage i.race           // base: white (race = 1)
regress wage ib3.race        // base: other (race = 3)
regress wage c.grade##i.union // returns to education by union status
regress wage c.ttl_exp##c.ttl_exp // quadratic in experience
```

Factor variables have three advantages over dummies created by hand: the base category is omitted automatically, which avoids the dummy variable trap (perfect collinearity with the constant); interactions and polynomials do not require new variables; and post-estimation commands such as `margins` (Chapter 9) know that `grade` and `c.grade#c.grade` are the same variable, so they compute marginal effects correctly. Factor variables must be non-negative integers: string variables must be `encoded` first.

Exercises

1. Load `auto.dta`. Create the variable `price_mpg` equal to the product of `price` and `mpg`, and the variable `ln_weight` equal to the natural logarithm of `weight`.
2. Create a variable `rep_cat` equal to “Poor” for `rep78` equal to 1 or 2, “Average” for 3, and “Good” for 4 or 5, using `recode` with value labels. Check that missing values of `rep78` remain missing.
3. For each car, compute the difference between its price and the average price of the cars with the same `rep78`.
4. Load `nls88.dta`. Create a dummy equal to 1 for women who work more than 40 hours per week, taking care of missing values in `hours`.
5. Create a variable with the quintiles of `wage`, and tabulate it against `collgrad`.
6. Regress `wage` on `i.occupation`. Which category is the base? Re-estimate the model using the most frequent occupation as base category.

Combining and Reshaping Data

The main commands used to combine datasets are `append` and `merge`. To change the structure of the data we use `reshape`, `collapse`, `contract`, `expand` and `fillin`. In all these commands, the dataset currently in memory is called the *master* dataset, and the dataset stored on disk is called the *using* dataset.

5.1 Preserve, Restore and Temporary Files

Many commands in this chapter replace the data in memory. Two tools make it possible to go back:

- `preserve` saves a copy of the data in memory; `restore` brings it back, discarding all changes made in between. Both must be executed in the same run of the do-file;
- `tempfile name` creates a temporary file, referred to as ‘`name`’, which is automatically deleted at the end of the do-file. It is used to save intermediate datasets without filling the project folder.

```
sysuse auto, clear
preserve
    keep if foreign == 1
    summarize price
restore                // back to all 74 cars
```

5.2 Append

The `append` command combines datasets vertically, by adding observations:

```
append using filename [filename ...] [, generate(newvar) nolabel force]
```

where `generate(newvar)` creates a variable indicating the source of each observation (0 for the master, 1 for the first using dataset, and so on), and `nolabel` prevents copying the value labels from the using dataset.

Variables are matched by name, and it is not necessary for all variables to be in both datasets. For instance, if `second.dta` contains the variables `a`, `b` and `d`, and `first.dta` only `a` and `b`, the observations coming from `first.dta` will have missing values for `d`. A variable that is numeric in one dataset and a string in the other causes an error.

```
sysuse auto, clear
keep if foreign == 0
tempfile domestic
save `domestic'

sysuse auto, clear
keep if foreign == 1
append using `domestic', generate(source)
tabulate foreign source
```

5.3 Merge

The `merge` command combines datasets horizontally, by adding variables. Observations are matched using one or more **key variables** (identifiers) that are present in both datasets:

```
merge 1:1 varlist using filename [, options]
merge m:1 varlist using filename [, options]
merge 1:m varlist using filename [, options]
```

where:

- **1:1** (one-to-one): the key variables uniquely identify the observations in both datasets, e.g., the same individuals observed in two files;
- **m:1** (many-to-one): the key variables uniquely identify the observations in the using dataset but not in the master, e.g., adding regional characteristics (using, one row per region) to individual data (master, many individuals per region);
- **1:m** (one-to-many): the opposite case.

Stata checks that the key variables really identify the observations as declared, and stops with an error if they do not. The most useful options are:

- `keepusing(varlist)`: adds only the listed variables from the using dataset;
- `keep(match master using)`: keeps only the observations with the listed results (see below);
- `assert(match master using)`: stops with an error if there are observations with other results;
- `generate(newvar)`: name of the variable indicating the result of the merge (default: `_merge`);
- `update` and `replace`: fill in missing values (and, with `replace`, overwrite non-missing values) in the master with the values in the using dataset.

The variable `_merge` indicates where each observation comes from:

- `_merge == 1` (*master only*): the observation is in the master dataset only;

- `_merge == 2` (*using only*): the observation is in the using dataset only;
- `_merge == 3` (*matched*): the observation is in both datasets;
- `_merge == 4` and `5`: only with `update`, matched observations in which missing values were updated (4) or in which the two datasets disagree (5).

```
* Using dataset: average wage by industry (one row per industry)
sysuse nls88, clear
collapse (mean) avg_wage_ind = wage, by(industry)
tempfile ind
save `ind'

* Master dataset: individuals (many per industry)
sysuse nls88, clear
merge m:1 industry using `ind'
tabulate _merge
drop _merge
generate rel_wage = wage / avg_wage_ind
```

note: always tabulate `_merge` and understand why some observations are not matched before dropping them: unmatched observations often reveal problems in the key variables (different spelling, codes, leading zeros). Never use `merge m:m`: it does not do what its name suggests and it almost always produces wrong data. When the keys identify observations in neither dataset, `joinby` forms all pairwise combinations.

5.4 Reshape

The same data can be stored in **wide** format (one row per unit, one column per period) or in **long** format (one row per unit and period). Panel data commands (Chapter 11) require the long format.

Wide			Long		
id	inc80	inc81	id	year	inc
1	5000	5500	1	80	5000
2	2000	2200	1	81	5500
			2	80	2000
			2	81	2200

```
reshape long stubnames, i(varlist) j(newvar)
reshape wide stubnames, i(varlist) j(existingvar)
```

where `stubnames` are the common parts of the names of the variables that vary over time (`inc` for `inc80`, `inc81`), `i()` contains the identifier of the unit, and `j()` the variable that contains (or will contain) the suffix, typically the time period.

```
webuse reshape1, clear
list
reshape long inc ue, i(id) j(year) // from wide to long
list
reshape wide inc ue, i(id) j(year) // and back to wide
```

tips: before `reshape long`, check with `isid` that the variables in `i()` uniquely identify the observations. Variables that do not vary over time (e.g., `sex`) must not be listed among the `stubnames`.

5.5 Collapse

`collapse` replaces the data in memory with a dataset of summary statistics, one observation for each group:

```
collapse clist [weight] [if exp] [in range] [, by(varlist) cw]
```

where `clist` is a list of statistics and variables:

- (stat) `varlist`: computes the statistic for each variable, keeping its name;
- (stat) `newvar = varname`: stores the statistic in a new variable;
- `stat` is one of `mean` (the default), `median`, `sd`, `sum`, `count` (number of non-missing observations), `min`, `max`, `p#` (`#`-th percentile, e.g., `p25`), `iqr`, `first`, `last`, `firstnm`, `lastnm` (first and last non-missing value), `percent`, `semean`.

The `by(varlist)` option specifies the groups over which the statistics are computed; without it, the result is a single observation. The `cw` option (casewise deletion) uses only the observations with no missing values in any of the variables; by default, each statistic uses all the non-missing values of its variable.

```
sysuse nlsw88, clear
preserve
    collapse (mean) wage hours (count) n = wage, by(industry)
list
restore
```

5.6 Contract

`contract` replaces the data in memory with a dataset of frequencies: one observation for each combination of values of the variables in `varlist`, with a variable containing the frequency of each combination.

```
contract varlist [weight] [if exp] [in range] [, freq(newvar) zero nomiss]
```

where `freq(newvar)` names the frequency variable (default: `_freq`), `zero` also includes the combinations with zero frequency, and `nomiss` drops the observations with missing values in `varlist`.

```
sysuse auto, clear
contract foreign rep78, zero
list
```

5.7 Expand

`expand` replaces each observation with n copies of itself, where n is the value of the expression rounded to the nearest integer. If n is smaller than 1 or missing, the observation is kept once (it is not deleted).

```
expand [=]exp [if exp] [in range] [, generate(newvar)]
```

For example, `expand 2` results in two copies of each observation (the original and one duplicate); the option `generate(newvar)` creates a variable equal to 0 for the original observations and 1 for the copies. `expand` is often used to turn a dataset of frequencies into a dataset of individual observations (`expand _freq`).

5.8 Fillin

`fillin` rectangularizes a dataset: it adds observations, with missing values, so that all combinations of the values of the variables in `varlist` exist. It also creates the variable `_fillin`, equal to 1 for the added observations and 0 otherwise.

```
fillin varlist
```

For example, in a panel of regions and years in which region 3 is not observed in 2020, `fillin region year` adds the observation for region 3 in 2020, with `_fillin == 1` and missing values in all other variables.

Exercises

1. Load `auto.dta`. Save the cars with `rep78` equal to 1, 2 or 3 in a temporary file, then load the data again, keep the cars with `rep78` equal to 4 or 5 and append the temporary file. How many observations does the final dataset have, and why is it not 74?
2. Load `auto2.dta`, keep `make` and `rep78` and save them in a temporary file. Load `auto.dta`, drop `rep78` and merge the temporary file using `make` as key variable. Which type of merge is it? Tabulate `_merge`.
3. Load `nls88.dta`. Create a dataset with one observation per occupation, containing the mean and the median wage and the number of women, and label all the new variables.
4. Merge the dataset created in Exercise 3 back into `nls88.dta`, and create a dummy equal to 1 for women earning more than the median wage of their occupation.
5. Load the dataset `webuse reshape1`. Reshape it to long format, compute the average income by year, and reshape it back to wide format.

Descriptive Statistics and Tests

Descriptive statistics summarize the sample and are the first step of every empirical analysis: they reveal coding errors, outliers and missing values, and they describe the population to which the results refer. In this chapter we use the dataset `nlsw88.dta`, which contains wages and characteristics of 2,246 US women in 1988.

6.1 Summarize

The `summarize` command (abbreviated `sum`) reports the number of non-missing observations, the mean, the standard deviation, the minimum and the maximum of numeric variables:

```
[by varlist:] summarize [varlist] [if exp] [in range] [weight] [, detail meanonly format]
```

where:

- `detail` also reports percentiles, the four smallest and largest values, the variance, the skewness and the kurtosis;
- `meanonly` computes the statistics without displaying them (useful together with stored results, Chapter 12); it cannot be combined with `detail`;
- `format` displays the statistics using the display format of each variable.

```
sysuse nlsw88, clear
summarize                               // all variables
summarize wage hours ttl_exp
summarize wage, detail
bysort union: summarize wage
```

note: look at the number of observations (`Obs`) for each variable: differences between variables reveal missing values, which will reduce the sample in any regression that uses those variables.

6.2 Frequency Tables: Tabulate

The `tabulate` command (abbreviated `tab`) produces one-way and two-way frequency tables:

```
[by varlist:] tabulate varname [if exp] [in range] [weight] [, options]
[by varlist:] tabulate varname1 varname2 [if exp] [in range] [weight] [, options]
```

The most useful options are:

- `missing`: treats missing values as a category and displays them in the table;
- `nolabel`: displays the numeric codes instead of the value labels;
- `generate(stub)`: creates one dummy for each category (Chapter 4);
- `sort`: (one-way tables) lists the categories in descending order of frequency;
- `row`, `column`, `cell`: (two-way tables) display the percentages within each row, within each column, and over the whole table;
- `nofreq`: (two-way tables) suppresses the frequencies, so that only the percentages are shown;
- `chi2`, `exact`, `V`: (two-way tables) Pearson's χ^2 test of independence between rows and columns, Fisher's exact test, and Cramér's V ;
- `matcell(name)`: saves the frequencies in a matrix.

```
tabulate race
tabulate union, missing
tabulate race union, row chi2          // % of union members by race, and test
tab1 race married collgrad            // one-way tables for several variables
```

The `summarize()` option of `tabulate` reports the mean, standard deviation and frequency of a variable for each category (or combination of categories):

```
tabulate race, summarize(wage)
tabulate race union, summarize(wage) means    // only the means
```

6.3 Tables of Statistics: Tabstat and Table

Tabstat

`tabstat` displays several statistics for several variables, optionally by groups:

```
tabstat varlist [if exp] [in range] [weight] [, statistics(statlist) by(varname)
        columns(statistics/variables) format(%fmt)]
```

Available statistics include `mean`, `sd`, `n` (count), `min`, `max`, `p50` (median) and any other percentile `p#`, `iqr`, `sum`, `semean`.

```
tabstat wage hours ttl_exp, statistics(n mean sd p50 min max) columns(statistics)
tabstat wage hours, by(union) statistics(mean sd n) format(%9.2f)
```

Table

`table` builds tables of frequencies and statistics with rows, columns and several statistics. Its syntax was completely redesigned in Stata 17¹:

```
table (rowvars) (colvars) [if exp] [in range] [weight] [, statistic(stat varlist)
      nformat(%fmt) nototals]
```

where the variables in the first parentheses define the rows and those in the second the columns. The option `statistic()` can be repeated to show several statistics; without it, the table reports frequencies. Totals are shown by default and removed with `nototals`.

```
table (race) (union)                                // frequencies
table (race) (union), statistic(mean wage) nformat(%9.2f)
table (race), statistic(mean wage) statistic(sd wage) statistic(frequency)
```

Descriptive tables for papers: `dtable`

From Stata 18, `dtable` produces the typical “Table 1” of a paper (means and standard deviations of continuous variables, frequencies and percentages of categorical variables, optionally by group) and exports it:

```
dtable wage hours ttl_exp i.race i.collgrad, by(union, tests) ///
      export("$Output/table1.docx", replace)
```

The format of the output file is determined by its extension (`.docx`, `.xlsx`, `.tex`, `.pdf`, ...).

6.4 Means and Confidence Intervals

The `ci` command reports means, proportions or variances with their standard errors and confidence intervals²:

```
ci means varlist [if exp] [in range] [weight] [, level(#)]
ci proportions varlist [if exp] [in range] [, level(#)]
```

where `level(#)` sets the confidence level (default: 95%). `ci proportions` requires 0/1 variables.

```
ci means wage hours
ci means wage, level(99)
ci proportions union collgrad
```

The `means` command reports the arithmetic, geometric and harmonic means with their confidence intervals. The geometric and harmonic means require positive values; the option `add(#)` adds a constant to all values before computing them.

¹Code written for Stata 16 or earlier (with options such as `contents()`, `row`, `col`) runs only under version control: `version 16: table ...`

²Up to Stata 13 the syntax was `ci varlist`.

6.5 Percentiles and Cumulative Distribution

`centile` reports percentiles with their confidence intervals:

```
centile [varlist] [if exp] [in range] [, centile(numlist) cci normal meansd level(#)]
```

where `centile(numlist)` lists the percentiles (default: the median). By default, the confidence intervals are obtained with a binomial method that makes no assumption on the distribution of the variable; `cci` computes conservative (non-interpolated) binomial intervals, `normal` assumes that the estimated percentiles are asymptotically normal, and `meansd` assumes that the variable itself is normally distributed.

```
centile wage, centile(10 25 50 75 90)
```

Other related commands:

- `xtile newvar = exp, nquantiles(#)` creates a variable with the quantile group of each observation (Chapter 4);
- `pctile newvar = exp, nquantiles(#)` creates a variable containing the cut-off points of the quantiles, stored in the first $\#-1$ observations;
- `cumul varname, generate(newvar)` creates a variable containing the empirical cumulative distribution function of `varname`, normalized to 1 at the largest value (`freq` gives it in frequencies, `by()` computes it by group; only `fweight` and `aweight` are allowed).

```
cumul wage, generate(cdf_wage)
twoway line cdf_wage wage, sort           // plot of the empirical CDF
```

6.6 Correlation

`correlate` reports the correlation matrix (or, with `covariance`, the covariance matrix) of a list of variables:

```
[by varlist:] correlate [varlist] [if exp] [in range] [weight] [, means covariance]
pworth [varlist] [if exp] [in range] [weight] [, obs sig star(#)]
```

`correlate` uses only the observations with no missing values in any of the variables (listwise deletion), so all correlations refer to the same sample. `pworth` computes each correlation on all the observations available for that pair (pairwise deletion), and with `sig` and `obs` reports the p-value of the test that the correlation is zero and the number of observations. Only `fweight` and `aweight` are allowed.

```
correlate wage hours ttl_exp tenure
pworth wage hours ttl_exp tenure, sig obs
```

6.7 Tests on Means and Proportions

t-tests

```

ttest varname == #                (one-sample)
ttest varname1 == varname2        (paired)
ttest varname, by(groupvar) [unequal] (two independent samples)

```

where:

- the one-sample test compares the mean of `varname` with the value `#`;
- the paired test compares the means of two variables measured on the same units (e.g., before and after an intervention);
- the two-sample test compares the mean of `varname` between the two groups defined by `groupvar`; `unequal` does not assume equal variances in the two groups (Welch test).

The output reports the p-values of three alternative hypotheses: that the difference is smaller than zero, different from zero (two-sided), and larger than zero.

```

ttest wage == 7
ttest wage, by(union) unequal

```

Tests on proportions

`prtest` tests hypotheses on proportions of 0/1 variables, with the same syntax as `ttest`:

```
prtest collgrad, by(union)
```

Hotelling's T^2

`hotelling varlist, by(groupvar)` tests whether the means of several variables are jointly equal between two groups. It is the multivariate version of the two-sample t-test, and is used, for example, to check the balance of covariates between treated and control units.

```
hotelling age grade ttl_exp, by(union)
```

Exercises

1. Load `auto.dta`. Count the number of cars with a price greater than \$10,000.
2. Report detailed summary statistics for `mpg`. Is the distribution skewed? Compare the mean and the median.
3. Create a two-way table of `rep78` and `foreign`, with column percentages. Include the missing values of `rep78`.
4. Using `tabstat`, report the mean, the median, the standard deviation and the number of observations of `price`, `mpg` and `weight`, separately for domestic and foreign cars.
5. Compute the 95% and the 99% confidence intervals of the mean of `price`.
6. Test whether the mean price of domestic and foreign cars is the same, with and without assuming equal variances.
7. Load `nlsw88.dta` and produce a descriptive table of `wage`, `hours`, `t1l_exp`, `race` and `collgrad` by `union`, exported to a Word file.

Data visualization is an essential step in data analysis: it helps to explore the data, to check assumptions and to present results. Stata's graphs are produced by a few main commands, all sharing the same options for titles, axes, legends and colours.

7.1 Histograms and Densities

A histogram displays the distribution of a single continuous variable:

```
sysuse nlsw88, clear
histogram wage
histogram wage, percent width(2)           // bars of width 2 dollars, in %
histogram wage, normal                     // overlays a normal density
histogram race, discrete percent           // discrete variable: one bar per value
histogram wage, by(union)                  // one panel per group
```

where `width()` or `bin()` set the width or the number of the bars, `percent`, `fraction` and `frequency` change the units of the vertical axis (default: density), and `discrete` is used for variables that take few integer values.

A kernel density is a smoothed version of the histogram, and is convenient to compare distributions:

```
kdensity wage
twoway (kdensity wage if union == 1) (kdensity wage if union == 0), ///
      legend(order(1 "Union" 2 "Non-union"))
```

7.2 Twoway Graphs: Relationships Between Variables

The `twoway` command is the umbrella for scatter plots, line plots and fitted lines. Several plots can be overlaid by writing each of them in parentheses:

```
twoway (plottype yvar xvar [if] [in] [, plot_options]) (...) [, graph_options]
```

In every `twoway` plot, the **first** variable goes on the vertical axis and the **last** on the horizontal axis.

Scatter plots and fitted lines

```
sysuse auto, clear
twoway (scatter price mpg)           // price on y, mpg on x
twoway (scatter price mpg) (lfit price mpg) // with the OLS line
twoway (scatter price mpg) (qfit price mpg) // with a quadratic fit
twoway (scatter price mpg) (lfitci price mpg) // OLS line with 95% CI
```

Line plots

`line` connects the points in the order of the data, so the data must be sorted by the variable on the horizontal axis (or the option `sort` must be used). It is typically used for time series:

```
webuse nlswork, clear
preserve
    collapse (mean) ln_wage, by(year)
    twoway (line ln_wage year), ytitle("Mean log wage") xtitle("Year")
restore
```

7.3 Bar Charts

Bar charts display summary statistics (by default, the mean) by category:

```
sysuse auto, clear
graph bar (mean) price, over(foreign) ///
    ytitle("Mean price (USD)") title("Mean price by car origin")
graph bar (mean) price mpg, over(foreign) // two statistics
graph hbar (count) price, over(rep78) // horizontal bars, counts
```

7.4 Titles, Axes and Legends

The following options are common to all graphs:

- `title("text"), subtitle("text"), note("text")`: titles and notes;
- `ytitle("text"), xtitle("text")`: axis titles (by default, the variable labels);
- `ylabel(0(5000)15000), xlabel(10(5)40)`: ticks and labels of the axes, from 0 to 15,000 every 5,000;
- `yscale(log)`: logarithmic scale;
- `legend(order(1 "label 1" 2 "label 2"))` or `legend(off)`: legend;
- `by(varname)`: one panel for each value of `varname`;
- `scheme(name)`: the general look of the graph (from Stata 18 the default scheme is `stcolor`; `s2color` is the old default, `s1mono` is black and white).

7.5 Graphs of Estimation Results

Graphs are often used to present regression results. Three commands are especially useful:

- `marginsplot`, after `margins`, plots predicted values or marginal effects with their confidence intervals (Chapter 9);
- `coefplot` (user-written: `ssc install coefplot`) plots the estimated coefficients with their confidence intervals;
- `binscatter` (user-written: `ssc install binscatter`) divides the horizontal axis into bins and plots the mean of y in each bin, which is more readable than a scatter plot with many observations.

```
sysuse nlsw88, clear
regress wage grade ttl_exp i.race i.union
coefplot, drop(_cons) xline(0)
binscatter wage ttl_exp, nquantiles(20)
```

7.6 Saving, Combining and Exporting Graphs

- `graph save "file.gph"`, `replace` saves the graph in Stata's format, so that it can be opened and edited later in Stata;
- `graph export "file.pdf"`, `replace` exports the graph currently displayed; the format is determined by the extension. PDF, EPS and SVG are vector formats, recommended for L^AT_EX; PNG is recommended for Word and PowerPoint (use `width(2000)` for a good resolution);
- `graph combine g1 g2` puts several graphs, saved in memory with the option `name()`, in the same figure.

```
sysuse auto, clear
histogram price, name(g1, replace)
histogram mpg, name(g2, replace)
graph combine g1 g2, cols(2)
graph export "$Graphs/hist_price_mpg.pdf", replace
graph export "$Graphs/hist_price_mpg.png", replace width(2000)
```

Exercises

1. Using the dataset `auto.dta`, create a histogram for the variable `weight`, with a normal density overlaid.
2. Create a scatter plot with `price` on the vertical axis and `mpg` on the horizontal axis. Then add a linear fitted line (`lfit`), and give the graph a title and axis titles.
3. Create a bar chart of the mean of `weight` by `foreign`.
4. Export the graph of Exercise 2 to a PDF file named `price_mpg_fit.pdf` in the `Graphs` folder of your project.
5. Create separate histograms of `length` for domestic and foreign cars using the `by()` option.
6. Load `nls88.dta` and compare the kernel densities of `wage` for college graduates and non-graduates in the same graph, with a legend.

Linear Regression

Estimation commands share a common syntax:

```
command depvar [indepvars] [if exp] [in range] [weight] [, options]
```

where **depvar** is the dependent variable and **indepvars** the list of independent variables (regressors), which can include factor variables (Section 4.5). The **if** and **in** qualifiers restrict the estimation sample. After estimation, the results are stored in memory until the next estimation command, and can be used by post-estimation commands such as **predict**, **test**, **lincom** and **margins**.

8.1 OLS Regression

The **regress** command (abbreviated **reg**) estimates a linear regression by ordinary least squares:

```
regress depvar [indepvars] [if exp] [in range] [weight] [, vce(vcetype) level(#)  
noconstant]
```

where **vce()** sets the type of standard errors (Section 8.4), **level(#)** the confidence level of the intervals (default: 95%), and **noconstant** removes the constant. For example:

```
sysuse nlsw88, clear  
regress wage grade  
regress wage grade ttl_exp tenure  
regress wage grade ttl_exp tenure if south == 1      // only women in the South  
regress wage grade ttl_exp tenure i.race i.union      // with categorical variables
```

8.2 Reading the Output

The output of **regress** has three parts.

Analysis of variance (top left). The total sum of squares of y (**Total**) is decomposed into the part explained by the model (**Model**, *SSE*) and the unexplained part (**Residual**, *SSR*),

with the corresponding degrees of freedom (**df**) and mean squares (**MS** = **SS**/**df**).

Summary statistics (top right).

- **Number of obs**: the number of observations used. Observations with missing values in any of the variables are excluded, so compare this number with the size of the dataset;
- **F(k, n-k-1)** and **Prob > F**: the F statistic and the p-value of the test that all slope coefficients are jointly equal to zero;
- **R-squared**: the share of the variance of y explained by the model, SSE/SST ;
- **Adj R-squared**: the R^2 adjusted for the number of regressors, which (unlike R^2) can decrease when a regressor is added;
- **Root MSE**: the estimate of the standard deviation of the error term, $\hat{\sigma}$.

Coefficient table. One row for each regressor and one for the constant (**_cons**):

- **Coefficient**: the OLS estimate $\hat{\beta}_j$;
- **Std. err.**: its standard error, $se(\hat{\beta}_j)$;
- **t**: the t statistic of the test $H_0 : \beta_j = 0$, equal to $\hat{\beta}_j/se(\hat{\beta}_j)$;
- **P>|t|**: the two-sided p-value of the same test. We reject H_0 at the 5% level if the p-value is below 0.05;
- **[95% conf. interval]**: the 95% confidence interval for β_j , approximately $\hat{\beta}_j \pm 1.96 se(\hat{\beta}_j)$ in large samples.

note: a coefficient that is statistically significant is not necessarily large, and a coefficient that is not significant is not necessarily zero. Always read the size of the coefficient (in the units of y and x) and the confidence interval, not only the p-value.

8.3 Interpreting the Coefficients

The interpretation of β_j depends on how y and x enter the model. Holding all other regressors constant:

Model	Specification	Interpretation of β
level-level	$y = \alpha + \beta x$	$\Delta x = 1 \Rightarrow \Delta y = \beta$ units
log-level	$\ln y = \alpha + \beta x$	$\Delta x = 1 \Rightarrow \Delta y \approx 100\beta \%$
level-log	$y = \alpha + \beta \ln x$	$\Delta x = 1\% \Rightarrow \Delta y \approx \beta/100$ units
log-log	$\ln y = \alpha + \beta \ln x$	$\Delta x = 1\% \Rightarrow \Delta y \approx \beta \%$ (elasticity)

In the log-level model, the approximation 100β is accurate only for small β ; the exact percentage change is $100(e^\beta - 1)$. For a dummy variable D in a log-level model, $100(e^\beta - 1)$ is the percentage difference in y between the group with $D = 1$ and the group with $D = 0$.

```
generate ln_wage = ln(wage)
regress ln_wage grade ttl_exp tenure i.union
display 100*(exp(_b[1.union]) - 1)           // exact % wage gap of union members
```

Other common cases:

- **Categorical variables** (`i.race`): each coefficient is the difference in y between that category and the base category;
- **Quadratic terms** (`c.ttl_exp##c.ttl_exp`): the effect of x is not constant, $\partial y / \partial x = \beta_1 + 2\beta_2 x$, and must be evaluated at some value of x (e.g., with `margins`, Chapter 9);
- **Interactions** (`c.grade##i.union`): the coefficient on `grade` is its effect for non-union members (`union=0`), and the coefficient on `1.union#c.grade` is the *difference* in the effect of `grade` between union and non-union members.

8.4 Standard Errors

By default, `regress` computes standard errors under the assumption of homoskedastic errors. The option `vce()` changes this assumption:

- `vce(robust)` (or simply `robust`): heteroskedasticity-robust standard errors (White/Hu-ber, with a small-sample correction, known as HC1). `vce(hc3)` is a variant that performs better in small samples;
- `vce(cluster clustvar)`: standard errors robust to heteroskedasticity *and* to correlation of the errors within the groups defined by `clustvar` (e.g., students in the same class, individuals observed in several years, firms in the same region). The number of clusters, not the number of observations, determines their reliability: with fewer than about 30–50 clusters they can be unreliable.

```
regress ln_wage grade ttl_exp tenure, vce(robust)
regress ln_wage grade ttl_exp tenure, vce(cluster industry) // 12 clusters: too few
```

The coefficients are identical across the three regressions; only the standard errors, and therefore t , p -values and confidence intervals, change. In cross-sectional data, robust standard errors are the standard choice.

8.5 Predicted Values and Residuals

After an estimation command, `predict` creates new variables with predictions and residuals:

```
predict newvar [if exp] [, xb residuals stdp stdf]
```

where `xb` (the default) gives the fitted values $\hat{y} = X\hat{\beta}$, `residuals` the residuals $\hat{u} = y - \hat{y}$, `stdp` the standard error of the linear prediction $x'\hat{\beta}$, i.e., of the estimated conditional mean $E[y|x]$, and `stdf` (after `regress` only) the standard error of the forecast of y for a new observation, which also includes the variance of the error term: $\text{stdf}^2 = \text{stdp}^2 + \hat{\sigma}^2$. `stdp` is used to build confidence intervals for the regression line, `stdf` to build prediction intervals for individual observations. `predict` computes the predictions for all observations with non-missing regressors, including those that were not used in the estimation. To restrict it to the estimation sample, use `if e(sample)`:

```
regress wage grade ttl_exp tenure
predict wage_hat if e(sample)
predict u_hat if e(sample), residuals
summarize wage wage_hat u_hat
```

note: after a regression of $\ln y$, `predict` gives $\widehat{\ln y}$, not $\hat{y}$. Taking $\exp(\widehat{\ln y})$ underestimates $E[y|x]$.

8.6 Hypothesis Testing After Estimation

Wald tests: `test`

The `test` command performs F (Wald) tests of linear hypotheses on the coefficients of the last estimated model. In `test`, a variable name stands for its coefficient; `_b[varname]` is equivalent.

```
sysuse nls88, clear
regress wage grade ttl_exp tenure i.race i.union, vce(robust)
test tenure // H0: coefficient of tenure = 0
test ttl_exp = tenure // H0: the two coefficients are equal
test ttl_exp tenure // H0: both coefficients are zero (joint)
test 2.race 3.race // H0: no differences across race groups
testparm i.race // same test, written with factor variables
test 2*ttl_exp + tenure = 1 // H0: a linear combination equals 1
```

The joint test of several coefficients is not equivalent to separate t -tests on each of them: two regressors can be individually not significant but jointly significant, typically when they are highly correlated.

Linear combinations: `lincom`

`lincom` estimates a linear combination of coefficients, with its standard error, t statistic and confidence interval. It is used when we want the size of an effect, not only a test:

```
lincom ttl_exp - tenure // difference between the two coefficients
lincom 10*grade // effect of 10 more years of education
```

`testnl` and `nlcom` do the same for nonlinear combinations of coefficients:

```
nlcom _b[ttl_exp]/_b[tenure] // ratio of the two coefficients
```

8.7 Diagnostics

- `estat hettest`: Breusch–Pagan test of heteroskedasticity (after `regress` without `vce(robust)`);
- `estat ovtest`: Ramsey RESET test of omitted nonlinear terms;
- `estat vif`: variance inflation factors, to detect multicollinearity (values above 10 are usually considered a warning);

- `rvfplot`: plot of the residuals against the fitted values, to detect heteroskedasticity and nonlinearity;
- `avplot varname`: added-variable plot, the partial relationship between y and one regressor (Frisch–Waugh–Lovell), useful to spot influential observations.

```
regress wage grade ttl_exp tenure
estat hettest
estat ovtest
estat vif
rvfplot, yline(0)
```

8.8 Stored Results

Estimation commands store their results in `e()`, which can be listed with `ereturn list` and used in later computations:

- `e(N)`: number of observations; `e(r2)`: R^2 ; `e(F)`: F statistic; `e(rmse)`: Root MSE;
- `_b[varname]` and `_se[varname]`: coefficient and standard error of `varname`;
- `e(sample)`: 1 for the observations used in the estimation, 0 otherwise;
- `e(b)` and `e(V)`: the vector of coefficients and the variance-covariance matrix.

```
regress wage grade ttl_exp tenure
ereturn list
display "t-statistic of grade: " _b[grade]/_se[grade]
summarize wage if e(sample)      // descriptive statistics on the estimation sample
```

8.9 Publication-Ready Tables

The standard Stata output is ideal for analysis, but not for papers and reports. Two tools produce tables that can be exported to L^AT_EX, Word or Excel.

Installing user-written commands (SSC)

The Statistical Software Components (SSC) archive hosts thousands of community-developed Stata programs. A package is installed once with `ssc install packagename` and then works like any built-in command.

`estout` and `esttab`

The `estout` package (Jann, 2007), and in particular its command `esttab`, compiles several stored estimation results into a single table:

```
ssc install estout, replace
sysuse nlsw88, clear
```

```

generate ln_wage = ln(wage)
eststo clear
eststo m1: regress ln_wage grade, vce(robust)
eststo m2: regress ln_wage grade ttl_exp tenure, vce(robust)
eststo m3: regress ln_wage grade ttl_exp tenure i.race i.union, vce(robust)

esttab m1 m2 m3, se star(* 0.10 ** 0.05 *** 0.01) r2 label // on screen
esttab m1 m2 m3 using "$Output/wage_reg.tex", replace ///
    se star(* 0.10 ** 0.05 *** 0.01) b(3) se(3) r2 label booktabs ///
    mtitles("Baseline" "Experience" "Full") nobaselevels

```

where `eststo` stores the results of each regression (like `estimates store`), `se` reports standard errors instead of t statistics in parentheses, `star()` sets the significance stars, `b(3)` and `se(3)` the number of decimals, `label` uses the variable labels instead of the names, `booktabs` formats the table for the L^AT_EX package `booktabs`, and `nobaselevels` omits the rows of the base categories. The extension of the file (`.tex`, `.rtf`, `.csv`) determines the format.

etable

From Stata 17, the built-in command `etable` does the same without installing anything:

```

quietly regress ln_wage grade, vce(robust)
estimates store m1
quietly regress ln_wage grade ttl_exp tenure, vce(robust)
estimates store m2
etable, estimates(m1 m2) mstat(N) mstat(r2) showstars showstarsnote ///
    export("$Output/wage_reg.docx", replace)

```

Exercises

1. Using the dataset `auto.dta`, estimate a linear regression with `price` as dependent variable and `mpg` and `weight` as independent variables. Interpret the coefficient of `weight`, the R^2 and the F test.
2. Estimate the same regression with heteroskedasticity-robust standard errors. What changes, and what does not?
3. Generate the predicted values and the residuals of the regression in Exercise 1, and check that the mean of the residuals is zero.
4. Add `foreign` to the regression and test whether `mpg` and `weight` are jointly significant.
5. Estimate a regression of $\ln(\text{price})$ on `mpg`, `weight` and `foreign`. Compute the exact percentage difference in price between foreign and domestic cars.
6. Load `nlsw88.dta`. Regress the log of `wage` on `grade`, experience (`ttl_exp`) and its square using factor-variable notation. Use `lincom` to compute the effect of one more year of experience for a woman with 10 years of experience.
7. Put the regressions of Exercises 1, 4 and 5 in a single table and export it to a $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ file.

Binary Outcome Models

When the dependent variable is binary (1 if an event happens, 0 otherwise), the regression models the probability of the event, $P(y = 1|x)$. In this chapter we use `nlsw88.dta` to study the probability that a woman is a union member.

9.1 Linear Probability Model

The linear probability model (LPM) is an OLS regression of the binary variable on the regressors. Each coefficient is the change in the **probability** that $y = 1$ (in percentage points, once multiplied by 100) associated with a unit change in the regressor. Since the variance of a binary variable depends on x , the errors are heteroskedastic by construction, and robust standard errors must be used:

```
sysuse nlsw88, clear
regress union grade ttl_exp i.race south, vce(robust)
```

The LPM is simple to estimate and to interpret, but the predicted probabilities can be smaller than 0 or larger than 1, and the effect of each regressor is constant. Logit and probit models solve these problems.

9.2 Logit and Probit

The logit and probit models assume $P(y = 1|x) = F(x'\beta)$, where F is the cumulative distribution function of the logistic distribution (logit) or of the standard normal distribution (probit). Both are estimated by maximum likelihood:

```
logit depvar [indepvars] [if exp] [in range] [weight] [, vce(vcetype) level(#) or]
probit depvar [indepvars] [if exp] [in range] [weight] [, vce(vcetype) level(#)]
```

where `depvar` must be coded 0/1 (more precisely, 0 and non-zero), and `vce(robust)` and `vce(cluster clustvar)` work as in `regress`.

```
logit union grade ttl_exp i.race south
probit union grade ttl_exp i.race south
```

The output reports the log likelihood at each iteration of the maximization, the likelihood-ratio test (LR chi2) that all slope coefficients are zero, and the Pseudo R2, which is *not* the share of explained variance and is not comparable with the R^2 of OLS. In the coefficient table, z replaces t because inference is based on the normal distribution.

note: the coefficients of logit and probit are **not** marginal effects. Only their sign and their statistical significance can be read directly. The magnitude of the effect of x_j on the probability is $\partial P / \partial x_j = f(x'\beta) \beta_j$, which depends on the values of all regressors. Logit coefficients are roughly 1.6 times probit coefficients, but the marginal effects of the two models are usually very similar.

9.3 Marginal Effects: margins

The `margins` command computes marginal effects and predicted probabilities after any estimation command¹:

```
margins [marginlist] [if exp] [in range] [, dydx(varlist) at(atspec) atmeans]
```

The two most common marginal effects are:

- **average marginal effect** (AME): the marginal effect is computed for each observation, at its own values of the regressors, and then averaged over the sample. It is the default of `margins`, `dydx()` and is usually the preferred measure;
- **marginal effect at the means** (MEM): the marginal effect computed for a hypothetical observation with all regressors at their sample means (option `atmeans`). With dummy regressors, “the mean” is not a real observation (e.g., 0.42 South).

```
logit union grade ttl_exp i.race south
margins, dydx(*)                // AME of all regressors
margins, dydx(grade) atmeans    // MEM of grade
margins, dydx(grade) at(south=(0 1)) // AME of grade, in and out of the South
```

For continuous regressors, `margins`, `dydx()` reports the derivative of the probability. For regressors entered as factor variables (`i.race`, and dummies written as `i.south`), it reports the **discrete change** in the probability when the variable goes from the base category to each other category, which is the correct measure for a dummy. This is a further reason to use factor-variable notation: if `south` is entered as a continuous variable, `margins` treats it as continuous.

Predicted probabilities

Without `dydx()`, `margins` reports predicted probabilities, averaged over the sample or at specified values:

```
logit union grade ttl_exp i.race i.south
margins i.race                // average predicted probability by race
margins, at(grade=(8(2)18))  // ... for different years of education
marginsplot                  // plot of the last margins
```

The predicted probability of each observation is obtained with `predict newvar, pr`.

¹`margins` replaced `dprobit` and `mfx` in Stata 11. The two old commands are no longer documented and continue to work only for backward compatibility: they should not be used in new code.

9.4 Odds Ratios

In the logit model, $\exp(\beta_j)$ is the **odds ratio**: the factor by which the odds, $P/(1 - P)$, are multiplied when x_j increases by one unit. The option `or` of `logit`, or the command `logistic`, reports odds ratios instead of coefficients:

```
logit union grade ttl_exp i.race i.south, or
```

An odds ratio of 1.10 means that the odds increase by 10%, which is not the same as a 10% (or 10 percentage point) increase in the probability.

9.5 Comparing the Three Models

In most applications, the average marginal effects of LPM, logit and probit are very close. The table below compares them:

```
eststo clear
eststo lpm: regress union grade ttl_exp i.race i.south, vce(robust)
quietly logit union grade ttl_exp i.race i.south, vce(robust)
eststo logit: margins, dydx(*) post
quietly probit union grade ttl_exp i.race i.south, vce(robust)
eststo probit: margins, dydx(*) post
esttab lpm logit probit, se nobaselevels mtitles("LPM" "Logit AME" "Probit AME")
```

The option `post` makes the results of `margins` replace the estimation results in `e()`, so that they can be stored and tabulated like coefficients.

Exercises

1. Load `auto.dta`. Estimate a linear probability model of `foreign` on `price`, `mpg` and `weight`, with robust standard errors. How many predicted values fall outside the interval $[0, 1]$?
2. Estimate the same model with logit and with probit. Compare the signs and the significance of the coefficients.
3. After the logit, compute the average marginal effects and the marginal effects at the means. Interpret the average marginal effect of `weight` (hint: express it for a 100-pound increase).
4. Compute the predicted probability of being foreign for cars with `mpg` equal to 15, 20, 25, 30 and 35, and plot it with `marginsplot`.
5. Load `nlsw88.dta`. Estimate a logit of `collgrad` on `i.race`, `i.south` and `i.married`, and report the odds ratios. Then compute the discrete change in the probability of being a college graduate for women living in the South.

Instrumental Variables

Instrumental variable (IV) regression is used when one or more explanatory variables are **endogenous**, i.e., correlated with the error term (because of omitted variables, measurement error or simultaneity). In this case OLS is biased and inconsistent. An instrument z must satisfy two conditions:

1. **relevance**: z is correlated with the endogenous variable, conditional on the exogenous regressors. This condition can be tested;
2. **exogeneity**: z is uncorrelated with the error term. It is useful to think of it as the combination of two distinct assumptions:
 - *independence*: z is as good as randomly assigned, i.e., unrelated to the unobserved determinants of y (conditional on the exogenous regressors);
 - *exclusion restriction*: z affects y only through the endogenous variable, and has no direct effect on y .

In the linear model both are summarized by $Cov(z, u) = 0$, but they fail for different reasons, and both must be argued on economic grounds: neither can be tested.

10.1 Two-Stage Least Squares: `ivregress`

```
ivregress 2sls depvar [exogvars] (endogvars = instruments) [if exp] [in range] [, first  
vce(vcetype)]
```

where:

- **depvar** is the dependent variable;
- **exogvars** are the exogenous regressors (controls). They are automatically used as instruments for themselves;
- **endogvars** are the endogenous regressors;
- **instruments** are the excluded instruments, i.e., the variables that enter the first stage but not the equation of y . There must be at least as many instruments as endogenous variables;

- **first** displays the first-stage regression(s), and `vce(robust)` or `vce(cluster clustvar)` work as in `regress`.

For example, suppose you want to estimate the effect of education on wages, but education is endogenous (e.g., because of unobserved ability). With experience and gender as controls and the distance to the nearest college as instrument, the command would be:

```
ivregress 2sls ln_wage experience female ///
      (education = distance_college), first vce(robust)
```

The validity of this instrument rests on two assumptions: that families living close to a college do not differ in other determinants of wages (independence), and that distance affects wages only through education, and not, for example, through the local labour market (exclusion). Both are debatable. Commonly used alternatives, such as parental education, are even more problematic: parents' education is correlated with the ability of their children (independence fails) and affects wages also through networks and wealth (exclusion fails). Discussing these assumptions is the most important part of any IV analysis.

A runnable example uses data on housing in the 50 US states in 1980: the median rent is regressed on the median value of houses (endogenous) and on the share of urban population, using family income and region dummies as instruments for the value of houses.

```
webuse hsng2, clear
regress rent pcturban hsngval // OLS
ivregress 2sls rent pcturban (hsngval = faminc i.region), first vce(robust)
```

10.2 What 2SLS Does

The name comes from two stages:

1. in the **first stage**, the endogenous variable is regressed on the instruments and on the exogenous regressors, and its predicted value is computed. The prediction is the part of the endogenous variable that is explained by the instruments, and it is therefore exogenous;
2. in the **second stage**, y is regressed on the predicted value of the endogenous variable and on the exogenous regressors.

note: running the two stages by hand with `regress` gives the correct coefficients but **wrong standard errors**, because the second stage treats the predicted value as if it were data. Always use `ivregress`.

10.3 Post-Estimation Tests

Weak instruments: `estat firststage`

`estat firststage` reports, for each endogenous variable, the first-stage R^2 , the partial R^2 of the excluded instruments and the F statistic of the test that their coefficients are jointly zero. When the instruments are **weak**, IV estimates are biased towards OLS and the usual confidence intervals are unreliable. How to judge the strength of the instruments depends on the setting:

- **one endogenous variable, homoskedastic errors:** a first-stage F below 10 is the traditional warning (Staiger and Stock, 1997). Stata also reports the minimum eigenvalue statistic with the Stock–Yogo critical values. Recent research suggests that 10 is too low for reliable t -based inference (Andrews, Stock and Sun, 2019);
- **robust or clustered standard errors:** after `vce(robust)` or `vce(cluster)`, `estat firststage` reports a robust F , but the Stock–Yogo critical values are derived under homoskedasticity and do not apply. The appropriate statistic is the effective F of Montiel Olea and Pflueger (2013), computed by the user-written command `weakivtest` (`ssc install weakivtest`);
- **several endogenous variables:** the F of each first stage is not sufficient, because the instruments may predict all endogenous variables through the same combination. Stata reports the Cragg–Donald minimum eigenvalue statistic (homoskedastic case); `ivreg2` reports the Kleibergen–Paap statistic (robust case) and the Sanderson–Windmeijer conditional F for each endogenous variable.

Endogeneity and overidentification

- **estat endogenous:** test of the null hypothesis that the endogenous variables are in fact exogenous. If it is not rejected, OLS and IV are not statistically different, and OLS is more efficient;
- **estat overid:** test of overidentifying restrictions, available only when there are more instruments than endogenous variables. It tests whether the different instruments give the same estimates; not rejecting it does not prove that the instruments are valid, since all of them could be invalid in the same way.

The test that Stata reports depends on the estimator and on the type of standard errors:

Estimator	<code>estat endogenous</code>	<code>estat overid</code>
2SLS, default VCE	Durbin; Wu–Hausman	Sargan; Basman
2SLS, <code>vce(robust)</code> or <code>vce(cluster)</code>	Wooldridge robust score; robust regression-based	Wooldridge robust score
GMM (<code>ivregress gmm</code>)	C (difference-in-Sargan)	Hansen’s J
LIML (<code>ivregress liml</code>)	not available	Anderson–Rubin; Basman F

The Sargan and Durbin–Wu–Hausman tests are valid only under homoskedasticity. With heteroskedastic or clustered errors, use the robust versions, which Stata reports automatically when the model is estimated with `vce(robust)` or `vce(cluster)`.

```
ivregress 2sls rent pcturban (hsngval = faminc i.region), vce(robust)
estat firststage           // robust F
estat endogenous           // Wooldridge robust score and regression-based tests
estat overid               // Wooldridge robust score test
```

tips: `ivregress` also estimates the model by limited-information maximum likelihood (`ivregress liml`), which is less biased than 2SLS with weak instruments, and by GMM (`ivregress gmm`). The user-written command `ivreg2` (`ssc install ivreg2`) reports a wider set of weak-instrument diagnostics.

Exercises

1. Load `webuse hsng2`. Estimate the OLS regression of `rent` on `pcturban` and `hsngval`, and the 2SLS regression using `faminc` and `i.region` as instruments for `hsngval`. Compare the coefficients of `hsngval`. In which direction is the OLS estimate biased, and why could this be?
2. Report the first stage and the first-stage F statistic. Are the instruments strong?
3. Run the two stages by hand with `regress` and `predict`. Compare the coefficients and the standard errors with those of `ivregress`.
4. Test whether `hsngval` is endogenous and whether the overidentifying restrictions are rejected.
5. Discuss whether family income is a credible instrument for the value of houses in a regression of rents.

Panel (longitudinal) data observe the same units (individuals, firms, municipalities) in several periods. In this chapter we use `nlswork.dta`, a panel of young US women interviewed between 1968 and 1988. Panel data must be in long format: one observation for each unit and period (see `reshape` in Chapter 5).

11.1 Declaring and Describing a Panel

```
xtset panelvar [timevar]
```

`xtset` declares the variable that identifies the units and the variable that identifies the time periods. The combination of the two must uniquely identify the observations.

```
webuse nlswork, clear
isid idcode year
xtset idcode year
xtdescribe                // patterns of participation over time
xtsum ln_wage grade ttl_exp // overall, between and within variation
```

`xtsum` decomposes the standard deviation of each variable into the *between* component (variation across units of the unit means) and the *within* component (variation over time around each unit's mean). A variable with no within variation, such as `race`, cannot be used in a fixed-effects regression.

11.2 Time-Series Operators

After `xtset` (or `tsset` for a single time series), the following operators can be used in any expression or variable list:

- `L.x`: lag of `x` (value in the previous period); `L2.x`: second lag;
- `F.x`: lead of `x` (value in the next period);
- `D.x`: first difference, $x - L.x$.

```
generate lag_wage = L.ln_wage
generate wage_growth = D.ln_wage
regress ln_wage L.ln_wage ttl_exp
```

Unlike `x[_n-1]` (Chapter 2), these operators take gaps into account: if a woman is observed in 1980 and 1982 but not in 1981, `L.ln_wage` in 1982 is missing, because the value of 1981 is not available.

11.3 Fixed and Random Effects

Consider the model $y_{it} = x'_{it}\beta + \alpha_i + \varepsilon_{it}$, where α_i is an unobserved, time-invariant characteristic of unit i (e.g., ability).

```
xtreg depvar [indepvars] [if exp] [in range] [, fe re vce(cluster panelvar)]
```

- **Fixed effects (fe):** the within estimator removes α_i by subtracting from each variable its unit mean. It is consistent even if α_i is correlated with the regressors, and β is identified only from the variation within each unit over time. The coefficients of time-invariant variables cannot be estimated;
- **Random effects (re):** a GLS estimator that assumes that α_i is uncorrelated with the regressors. Under this assumption it is more efficient than FE, and it can estimate the effect of time-invariant variables; if the assumption fails, it is inconsistent.

```
xtreg ln_wage ttl_exp tenure i.union i.year, fe vce(cluster idcode)
xtreg ln_wage ttl_exp tenure i.union i.year grade, re vce(cluster idcode)
```

The dummies `i.year` (time fixed effects) control for shocks common to all units in a given year, such as the business cycle. With panel data, the errors of the same unit are correlated over time, and standard errors should be clustered at the unit level.

In the FE output, `sigma_u` and `sigma_e` are the standard deviations of α_i and ε_{it} , `rho` is the share of the variance due to α_i , and the within R^2 is the relevant measure of fit.

Hausman test

The Hausman test compares FE and RE: under the null hypothesis that α_i is uncorrelated with the regressors, both are consistent and should give similar estimates. The standard test requires the models to be estimated without robust or clustered standard errors:

```
quietly xtreg ln_wage ttl_exp tenure i.union, fe
estimates store fe
quietly xtreg ln_wage ttl_exp tenure i.union, re
estimates store re
hausman fe re
```

A rejection of the null favours FE. In applied economics, FE is the default choice whenever the source of endogeneity is a time-invariant characteristic of the units.

tips: with several levels of fixed effects (e.g., worker and firm, or municipality and year) and many units, the user-written command `reghdfe` (`ssc install reghdfe`) is much faster than including dummies: `reghdfe ln_wage ttl_exp tenure, absorb(idcode year) vce(cluster idcode)`.

Exercises

1. Load `webuse nlswork`, declare the panel and report how many women are observed and in how many years on average.
2. Using `xtsum`, report the within and between standard deviations of `ln_wage`, `hours` and `grade`. Which of these variables could be used in a fixed-effects regression?
3. Create the one-year growth rate of wages with the `D.` operator. Why is it missing for some observations even after the first year of each woman?
4. Estimate the effect of `union` on `ln_wage`, controlling for `ttl_exp`, `tenure` and year dummies, with pooled OLS, fixed effects and random effects, clustering the standard errors by woman. Compare the three coefficients and discuss why they differ.
5. Run a Hausman test between fixed and random effects. Which model do you prefer?

Macros, Loops and Stored Results

This chapter introduces a few tools that make do-files shorter, easier to modify and less error-prone. The principle is simple: when the same operation is repeated, write it once.

12.1 Macros

A macro is a name that stands for a text. Every time Stata finds a macro in a command, it replaces it with its content before executing the command. There are two types:

- **local macros** exist only within the do-file (or the selection of lines) in which they are defined. They are defined with `local name content` and used as `'name'`, with a backtick (‘) at the beginning and an apostrophe (’) at the end;
- **global macros** exist until Stata is closed. They are defined with `global name content` and used as `$name`.

```
sysuse nlsw88, clear
local controls "ttl_exp tenure i.race i.south"
regress wage grade `controls'
regress wage grade `controls' if union == 1
display "The controls are: `controls'"
```

If the list of controls changes, it is modified only in one place. Locals are preferred to globals because they cannot be accidentally modified by another do-file; globals are used for information that must be shared across do-files, such as the paths of the project (Chapter 1).

note: local macros disappear when the execution stops. If you run only some lines of a do-file, the locals defined in lines that were not selected do not exist, and Stata replaces them with nothing, without an error. Many strange errors come from this.

tips: how to type the backtick ‘ on an Italian keyboard: on Mac press Option + \; on Windows hold Alt and type 96 on the numeric keypad.

The content of a macro can also be the result of an expression, if = is used:

```

local n = 10
local n_sq = `n'^2
display `n_sq'

```

12.2 Loops

foreach

`foreach` repeats the same commands for each element of a list:

```

foreach lname in list {
    commands using `lname'
}
foreach lname of varlist varlist {
    ...
}
foreach lname of local macroname {
    ...
}

```

where `lname` is a local macro that takes, one at a time, the value of each element of the list. The opening brace must be on the same line as `foreach`, and the closing brace on a line of its own. With `of varlist`, Stata checks that the elements are existing variables and expands abbreviations such as `x*`; with `of local`, the list is the content of a local macro.

```

sysuse auto, clear
foreach v of varlist price mpg weight length {
    generate ln_`v' = ln(`v')
    label variable ln_`v' "Log of `v'"
}

local outcomes "price mpg"
foreach y of local outcomes {
    regress `y' weight foreign, vce(robust)
}

```

forvalues

`forvalues` repeats the commands for a sequence of numbers:

```

forvalues k = 1/5 {
    display "Repair record `k': "
    count if rep78 == `k'
}
forvalues year = 1970(5)1990 {      // 1970, 1975, ..., 1990
    display `year'
}

```

levelsof

`levelsof` stores the distinct values of a variable in a local macro, which can then be used in a loop:

```

levelsof rep78, local(levels)
foreach l of local levels {
    quietly summarize price if rep78 == `l'
    display "rep78 = `l': mean price = " %9.2f r(mean)
}

```

12.3 Stored Results

Stata commands store their results in memory, where they can be used by later commands:

- **r-class** commands (e.g., `summarize`, `tabulate`, `count`, `correlate`, `ttest`) store results in `r()`, which can be listed with `return list`. They are overwritten by the next r-class command;
- **e-class** commands (estimation commands) store results in `e()`, listed with `ereturn list` (Chapter 8).

```

sysuse auto, clear
summarize price, detail
return list
generate price_std = (price - r(mean)) / r(sd)    // standardized price
count if foreign == 1
display "Share of foreign cars: " r(N)/_N

```

To keep a result, store it in a local macro or in a **scalar**:

```

quietly summarize price
scalar mean_price = r(mean)
local mean_price = r(mean)
display mean_price
display `mean_price'

```

note: stored results are overwritten by the next command of the same class. In the example above, after `count`, `r(mean)` of `summarize` no longer exists.

12.4 Conditional Execution: the if Command

The *if qualifier* (`command if exp`) selects observations. The *if command* executes a block of commands only if a condition is true, and is used with macros and scalars:

```

quietly count if missing(rep78)
if r(N) > 0 {
    display "rep78 has " r(N) " missing values"
}
else {
    display "rep78 has no missing values"
}

```

note: writing `if price > 5000 { ... }` does not select the cars with a price above 5,000: the `if` command evaluates `price` in the first observation only. To act on observations, use the `if` qualifier.

12.5 Debugging

- Run the do-file one block at a time, and check the results (with `count`, `tabulate`, `summarize`, `assert`) after each step that changes the data;
- `set trace on` displays every line executed inside loops and programs, with the macros already replaced by their content. It is turned off with `set trace off`;
- every error message comes with a return code, e.g., `r(111)`. Clicking on it in the results window opens its explanation. Appendix A lists the most common ones.

Exercises

1. Load `nlsw88.dta`. Store the list of controls `grade` `ttl_exp` `tenure` in a local macro and use it to estimate a regression of `wage` on the controls, separately for union and non-union members.
2. Using `foreach`, create the standardized version (mean 0, standard deviation 1) of `wage`, `hours` and `ttl_exp`, using the results stored by `summarize`.
3. Using `levelsof` and a loop, display the median wage for each industry.
4. Using `forvalues`, estimate the regression of `wage` on `grade` for each value of `race`, and display the coefficient of `grade` and its standard error each time.
5. Store the number of observations and the R^2 of a regression in two scalars, and display them in a sentence.

APPENDIX A

Common Errors

The table lists the error messages that students meet most often, their usual cause and the solution. Clicking on the return code in the results window opens a longer explanation.

Code	Message	Usual cause and solution
r(4)	no; dataset in memory has changed since last saved	Add the option <code>clear</code> to <code>use</code> , <code>sysuse</code> or <code>import</code> .
r(100)	varlist required	The command needs a list of variables.
r(109)	type mismatch	A string is used where a number is expected, or vice versa (e.g., <code>if make == 1</code> , or a number stored as string: use <code>destring</code>).
r(110)	variable already defined	<code>generate</code> on an existing variable: use <code>replace</code> , or drop the variable first.
r(111)	variable not found	Typo in the variable name, the variable was dropped, or a local macro is empty (lines run separately).
r(198)	invalid syntax	Typo, missing comma before the options, <code>=</code> instead of <code>==</code> , unbalanced parentheses or quotes, a command split over two lines without <code>///</code> .
r(199)	unrecognized command	Typo, or a user-written command that is not installed (<code>ssc install</code>).
r(601)	file not found	Wrong path or working directory: check with <code>pwd</code> and the globals of the project.
r(602)	file already exists	<code>save</code> or <code>export</code> without <code>replace</code> .
r(603)	file could not be opened	The file is open in another program (e.g., Excel) or the folder does not exist.
r(604)	log file already open	A previous run stopped before <code>log close</code> : add <code>capture log close _all</code> at the top.

Code	Message	Usual cause and solution
<code>r(2000)</code>	no observations	The <code>if</code> condition selects no observations, or all of them have missing values.
<code>r(459)</code>	variables do not uniquely identify observations	In <code>merge</code> or <code>reshape</code> : check the key variables with <code>isid</code> and <code>duplicates report</code> .
<code>r(9)</code>	assertion is false	An <code>assert</code> failed: the data are not as expected.

Silent errors

The most dangerous errors produce no error message:

- conditions with `>`, `>=` or `!=` that include missing values (Section 2.3);
- `//` used instead of `///` to split a command: the rest of the line is ignored;
- local macros that are empty because the line that defines them was not run;
- `merge` results that are not checked (`_merge`);
- results that depend on the order of tied observations after `sort`;
- random numbers generated without `set seed`.

Stata

Anderson, Mike – *An Introduction to Stata*.

Baum, Christopher F. (2006) – *An Introduction to Modern Econometrics Using Stata*. College Station, TX: Stata Press.

Cameron, A. Colin and Pravin K. Trivedi (2022) – *Microeconometrics Using Stata*, 2nd edition. College Station, TX: Stata Press.

Coda Moscarola, Flavia – *Mini STATA Course*.

Iezzi, Elisa – *STATA Tutorial: Primi passi con STATA. Gestione dati, grafici e semplici modelli*.

Jann, Ben (2007) – “Making regression tables simplified”. *The Stata Journal*, 7(2): 227–244.

Mitchell, Michael N. (2020) – *Data Management Using Stata: A Practical Handbook*, 2nd edition. College Station, TX: Stata Press.

Sianesi, Barbara – *Basic Stata*.

StataCorp – *Stata Base Reference Manual*. College Station, TX: Stata Press. Available from Stata with help and at <https://www.stata.com/manuals/>.

Econometrics

Andrews, Isaiah, James H. Stock and Liyang Sun (2019) – “Weak Instruments in Instrumental Variables Regression: Theory and Practice”. *Annual Review of Economics*, 11: 727–753.

Montiel Olea, José Luis and Carolin Pflueger (2013) – “A Robust Test for Weak Instruments”. *Journal of Business & Economic Statistics*, 31(3): 358–369.

Sanderson, Eleanor and Frank Windmeijer (2016) – “A Weak Instrument F -Test in Linear IV Models with Multiple Endogenous Variables”. *Journal of Econometrics*, 190(2): 212–221.

Staiger, Douglas and James H. Stock (1997) – “Instrumental Variables Regression with Weak Instruments”. *Econometrica*, 65(3): 557–586.

Stock, James H. and Motohiro Yogo (2005) – “Testing for Weak Instruments in Linear IV Regression”. In D. W. K. Andrews and J. H. Stock (eds.), *Identification and Inference for Econometric Models*. Cambridge: Cambridge University Press, 80–108.

Wooldridge, Jeffrey M. (2020) – *Introductory Econometrics: A Modern Approach*, 7th edition. Boston, MA: Cengage.

Online resources

UCLA Statistical Methods and Data Analytics, Stata resources: <https://stats.oarc.ucla.edu/stata/>

Stata cheat sheets (Tim Essam and Laura Hughes): <https://geocenter.github.io/StataTraining/>

Stata video tutorials: <https://www.youtube.com/user/statacorp>